

Full Marks:50

SET:A

Time: 1Hour 10Minutes

Q1. What is the major difference between the following?

```
import math
```

and

```
from math import sqrt
```

A)First provides the module namespace; second imports sqrt directly

B)There is no difference

C)Both import only sqrt

D)First imports only sqrt

Q2. What will be the output?

```
x = 10
```

```
def fun():
    print(x)
    x = 20
```

```
fun()
```

A)20

B)UnboundLocalError

C)NameError

D)10

Q3. What will normally happen?

```
import math
```

```
import math
```

A)The module is normally loaded and executed twice

B)SyntaxError

C)ImportError

D)The module is normally loaded once and reused

Q4. What will be printed?

```
try:
    x = int("10")
except ValueError:
    print("A")
```

```
else:
    print("B")
finally:
    print("C")
```

A)B

B)A C

C)B C

D)A

Q5. What is the output?

```
def fun(x):
    if x == 0:
        return
    return x + fun(x - 1)
```

```
print(fun(3))
```

A)6

B)3

C)5

D)None

Q6. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

A)Nothing

B)A and B

C)B

D)A

Q7. What is the output?

```
def fun(x):
    return x * 2
```

```
a = fun
```



print(a(5))

- A)10
- B)func(5)
- C)5
- D)Error

Q8. What is the purpose of this statement?

if __name__ == "__main__":

- A)It identifies code intended to run when the file is executed directly
- B)It prevents syntax errors
- C)It prevents all imports
- D)It creates the main function

Q9. What will be the output?

```
def fun(x):  
    try:  
        if x < 0:  
            raise ValueError("negative")  
        return x  
    except ValueError:  
        return 100  
    finally:  
        print("cleanup")
```

print(fun(-5))

- A)cleanup
- 100
- B)negative
- cleanup
- C)100
- cleanup
- D)cleanup
- 5

Q10. What will be printed?

```
try:  
    print(10 / 0)  
except ZeroDivisionError:  
    print("A")  
except Exception:  
    print("B")
```

- A)B
- B)A
- C)A B
- D)Nothing

Q11. What is the output?

```
def fun():  
    try:  
        print("A")  
        raise ValueError  
    except ValueError:  
        print("B")  
        return 10  
    else:  
        print("C")  
        return 20  
    finally:  
        print("D")
```

print(fun())

- A)A
- B
- D
- 10
- B)A
- C
- D
- 20
- C)A
- B
- C
- D
- 10
- D)A
- B
- 10
- D

Q12. What is the output?

```
try:  
    x = int("abc")  
except ValueError:  
    print("A")  
else:  
    print("B")  
finally:  
    print("C")
```

- A)B C
- B)A
- C)A C
- D)B



Q13. What happens here?

```
from math import sqrt
```

```
print(sqrt(36))
```

- A)6.0
- B)NameError
- C)6
- D)36

Q14. What happens when this code runs?

```
def first():  
    second()
```

```
def second():  
    third()
```

```
def third():  
    raise RuntimeError("Failure")
```

```
try:  
    first()  
except RuntimeError:  
    print("Handled")
```

- A)Handled
- B)RuntimeError cannot propagate through functions
- C)TypeError
- D)NameError

Q15. What is the output?

```
try:  
    print(10 / 0)  
except Exception:  
    print("A")  
except ZeroDivisionError:  
    print("B")
```

- A)A
- B)A B
- C)B
- D)SyntaxError

Q16. What will be printed?

```
def fun(x=[]):  
    x.append(10)  
    return x
```

```
a = fun()  
b = fun()
```

```
print(a)  
print(b)  
A)[10, 10]  
[10]  
B)Error  
C)[10]  
[10, 10]  
D)[10]  
[10]
```

Q17. What will be the output?

```
x = 10
```

```
def outer():  
    x = 20
```

```
    def inner():  
        global x  
        x = 30
```

```
    inner()  
    print(x)
```

```
outer()  
print(x)
```

- A)30
- 20
- B)20
- 30
- C)30
- 30
- D)20
- 10

Q18. What is the output?

```
def outer():  
    x = 10
```

```
    def inner():  
        return x + 5
```

```
    return inner
```

```
f = outer()  
print(f())
```



- A)10
- B)NameError
- C)15
- D)None

Q19. What happens?

```
def fun(a, b):
    return a + b
```

```
print(fun(10, a=20))
```

- A)SyntaxError
- B)30
- C)TypeError
- D)20

Q20. What will be printed?

```
def fun(a, b):
    return a + b
```

```
print(fun(10, b=20))
```

- A)SyntaxError
- B)TypeError
- C)20
- D)30

Q21. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```

- A)15
- B)5
- C>Error
- D)10

Q22. What happens here?

```
try:
    raise ValueError("Wrong value")
except ValueError as e:
    print(e)
```

- A)e
- B)ValueError
- C)Wrong Value
- D)Nothing

Q23. What will be printed?

```
def make_counter():
    count = 0
```

```
def counter():
    nonlocal count
    count += 1
    return count
```

```
    return counter
```

```
a = make_counter()
b = make_counter()
```

```
print(a())
print(a())
print(b())
print(a())
```

A)0

1

0

2

B)1

2

1

3

C)1

1

1

1

D)1

2

3

4

**Q24. if test.py has
print("A")**

```
if __name__ == "__main__":
    print("B")
```

If test.py is executed directly, what will be printed?

- A)A
- B)A followed by B
- C)B
- D)Nothing

Q25. What does the following function return?

```
def fun():
    print("Hello")
```

```
x = fun()
```



print(x)

- A)Hello None
- B)None
- C)Hello
- D)Error

Q26. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(
    map(
        lambda x: x * 2,
        filter(lambda x: x % 2 == 0,
numbers)
    )
)
```

print(result)

- A)[4, 8]
- B)[1, 3, 5]
- C)[2, 4, 6, 8, 10]
- D)[2, 4]

Q27. Which object keeps track of modules that have already been imported during the current Python process?

- A)__module__
- B)sys.path
- C)sys.modules
- D)sys.imports

Q28. What is the output?

```
try:
    raise ValueError
except ValueError:
    print("A")
    raise
```

- A)B
- B)No Output
- C)A, followed by propagation of the same exception
- D)A, and execution ends normally

Q29. What will be printed?

```
x = 10
```

```
def outer():
```

```
x = 20
```

```
def inner():
    print(x)
```

```
x = 30
inner()
```

outer()

- A)NameError
- B)30
- C)20
- D)10

Q30. What happens when this function is called?

```
def fun(a=10, b):
    return a + b
```

- A)SyntaxError
- B)TypeError
- C)NameError
- D)It executes correctly

Q31. What will be printed?

```
class InvalidAgeError(Exception):
    pass
```

```
def check_age(age):
    if age < 18:
        raise InvalidAgeError("Age is
too low")
    return "Allowed"
```

```
try:
    print(check_age(15))
except InvalidAgeError as e:
    print(type(e).__name__)
    print(e)
```

- A)InvalidAgeError
- Age is too low
- B)InvalidAgeError
- 15
- C)Exception
- Age is too low
- D)Nothing, because custom exceptions cannot be caught



Q32. What will be the output?

```
import math
```

```
print(math.sqrt(25))
```

- A)5
- B)sqrt(25)
- C)5.0
- D)25

Q33. What will happen?

```
def fun(a, b=10):  
    return a + b
```

```
print(fun(b=5, a=20))
```

- A)TypeError
- B)15
- C)SyntaxError
- D)25

Q34. Suppose sample.py contains:

```
x = 10
```

```
def display():  
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)  
sample.display()
```

- A)Only Hello
- B>Error
- C)x
display
- D)10
Hello

Q35. What will happen?

```
def outer():  
    x = 10
```

```
    def inner():  
        nonlocal y  
        y = 20
```

```
    inner()
```

```
outer()
```

- A)10
- B)SyntaxError
- C)NameError
- D)20

Q36. What will be printed?

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```

```
print(f())
```

- A)NameError
- B)None
- C)20
- D)10

Q37. What will be printed?

```
def fun():  
    print("A")  
    return  
    print("B")
```

```
fun()
```

- A)A B
- B)Nothing
- C)A
- D)B

Q38. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a  
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

- A)ImportError



B)10

10

C)Module loaded

Module loaded

10

D)Module loaded

10

Q39. What happens when this code executes?

```
def fun():  
    print(variable)  
    variable = 50
```

variable = 10

fun()

A)10

B)UnboundLocalError

C)50

D)NameError

Q40. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(  
    lambda a, b: a * b,  
    numbers  
)
```

```
print(result)
```

A)10

B)0

C)24

D)9

Q41. What will be printed?

```
def fun():  
    try:  
        return 10  
    finally:  
        print("Finally")
```

```
print(fun())
```

A)10 only

B)10 followed by Finally

C)Finally followed by 10

D)Finally only

Q42. What happens here?

```
class MyError(Exception):  
    pass
```

```
def test():  
    raise MyError("Something went  
wrong")
```

```
try:  
    test()  
except MyError as e:  
    print(type(e).__name__)
```

A)Something went wrong

B)MyError

C)TypeError

D)Exception

Q43. Why is

```
try:  
    print(10 / 0)  
except Exception:  
    print("A")  
except ZeroDivisionError:  
    print("B")  
codes's second handler never  
reached?
```

A)ZeroDivisionError occurs before
Exception

B)Exception is a superclass of
ZeroDivisionError

C)except blocks execute randomly

D)ZeroDivisionError is not an exception

Q44. What will happen?

```
def fun(n):  
    if n == 0:  
        return 1  
    return n * fun(n)
```

```
print(fun(3))
```

A)TypeError

B)RecursionError

C)6

D)3

Q45. What is the output?

```
def fun(x):
```



```
x += 10
return x
```

```
x = 5
y = fun(x)
```

```
print(x, y)
```

- A) 5 5
- B) 5 15
- C) 15 15
- D) 15 5

Q46. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```
import random
```

- A) Python automatically renames the local file
- B) random cannot be imported
- C) Python will always reject it
- D) The local file can shadow the intended standard-library module

Q47. What is the output?

```
def fun():
    try:
        return 10
    finally:
        return 20
```

```
print(fun())
```

- A) 10
- B) 20
- C) None
- D) RuntimeError

Q48. What is the output?

```
def outer():
    x = 10

    def inner():
        nonlocal x
        x = x + 5
        return x

    inner()
    return x
```

```
print(outer())
```

- A) 10
- B) 15
- C) 5
- D) UnboundLocalError

Q49. What is the output?

```
def fun(a=10, b=20):
    return a - b
```

```
print(fun(30))
```

- A) 20
- B) 10
- C) 30
- D) -10

Q50. What happens here?

```
def fun(a):
    return a
```

```
print(fun())
```

- A) None
- B) 0
- C) NameError
- D) TypeError

